In [1]:

```
# Importing all required libraries for the EDA project
import pandas as pd
import numpy as np
import statsmodels.api as sm
from bokeh.io import output_notebook
output_notebook()
import seaborn as sb
import matplotlib.pyplot as plt
from scipy import stats
from bokeh.io import show
import sys
import researchpy as rp
from bokeh.plotting import figure
```

BokehJS 2.3.2 successfully loaded.

In [2]:

```
# Creates a DataFrame to load the data in the file and preview it.

wage_df = pd.read_csv('wage.csv')
wage_df.head()
```

Out[2]:

	married	hourly_wage	years_in_education	years_in_employment	num_dependents	gender	race
0	1.0	3.24	12.0	2.0	3.0	female	white
1	0.0	3.00	11.0	0.0	2.0	male	white
2	1.0	6.00	8.0	28.0	0.0	male	white
3	1.0	5.30	12.0	2.0	1.0	male	white
4	1.0	8.75	16.0	8.0	0.0	male	white

In [47]:

```
# Encode marital status as a categorical variable

#category = pd.cut(wages_df.married, bins=[0,1], labels=['single', 'married'])
#wages_df['married'] = category.values
```

The dataset encodes marital status as **0 - Single** and **1 - married** hence we have to initially Encode the marital status as a categorical variable to enable us display accurate descriptive statistics.

1. Displaying Descriptive statistics on the dataset

In [4]:

```
# Displaying descriptive stats on the dataset

wage_df.describe(include='all')
```

Out[4]:

	married	hourly_wage	years_in_education	years_in_employment	num_dependents	gender	race
count	522.000000	517.000000	522.000000	519.000000	520.000000	521	515
unique	NaN	NaN	NaN	NaN	NaN	2	2
top	NaN	NaN	NaN	NaN	NaN	male	white
freq	NaN	NaN	NaN	NaN	NaN	272	461
mean	0.609195	5.917737	12.557471	5.152216	1.044231	NaN	NaN
std	0.488399	3.699058	2.757219	7.257133	1.258484	NaN	NaN
min	0.000000	0.530000	0.000000	0.000000	0.000000	NaN	NaN
25%	0.000000	3.350000	12.000000	0.000000	0.000000	NaN	NaN
50%	1.000000	4.670000	12.000000	2.000000	1.000000	NaN	NaN
75%	1.000000	6.880000	14.000000	7.000000	2.000000	NaN	NaN
max	1.000000	24.980000	18.000000	44.000000	6.000000	NaN	NaN

In [5]:

```
# Display relevant information on the dataset.

wage_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 525 entries, 0 to 524
Data columns (total 7 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
```

```
 0   married              522 non-null    float64
 1   hourly_wage          517 non-null    float64
 2   years_in_education   522 non-null    float64
 3   years_in_employment  519 non-null    float64
 4   num_dependents       520 non-null    float64
 5   gender               521 non-null    object
 6   race                 515 non-null    object
dtypes: float64(5), object(2)
memory usage: 28.8+ KB
```

2. Checking if any records in the data have any missing values; handling the missing data as appropriate.

In [6]:

```
# Returns number of empty/null values in the dataset.

print(wage_df.isnull().sum())
```

```
married                 3
hourly_wage             8
years_in_education      3
years_in_employment     6
num_dependents          5
gender                  4
race                   10
dtype: int64
```

In [7]:

```
# Replace empty/null numerical variables with the mean.

data = wage_df
data['hourly_wage'] = data['hourly_wage'].fillna(data.hourly_wage.mean())
data['years_in_education'] = data['years_in_education'].fillna(data.years_in_education.mean())
data['years_in_employment'] = data['years_in_employment'].fillna(data.years_in_employment.mean())
data['num_dependents'] = data['num_dependents'].fillna(data.num_dependents.mean())
```

In [8]:

```
# Printing results
print(wage_df.isnull().sum())
```

```
married                 3
hourly_wage             0
years_in_education      0
years_in_employment     0
```

```
num_dependents           0
gender                   4
race                    10
dtype: int64
```

In [9]:

```
# Replace missing gender values with the modal value

gender_mode = data.gender.mode()[0]
data.gender.fillna(gender_mode, inplace=True)
race_mode = data.race.mode()[0]
data.race.fillna(race_mode, inplace=True)
```

In [10]:

```
# Replacing the empty cells with the modal value
data.married.fillna(data.married.mode()[0], inplace=True)
```

In [12]:

```
print(wage_df.isnull().sum())
```

```
married                 0
hourly_wage             0
years_in_education      0
years_in_employment     0
num_dependents          0
gender                  0
race                    0
dtype: int64
```

In [13]:

```
# displaying the information about the datase
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 525 entries, 0 to 524
Data columns (total 7 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   married              525 non-null    float64
 1   hourly_wage          525 non-null    float64
 2   years_in_education   525 non-null    float64
 3   years_in_employment  525 non-null    float64
 4   num_dependents       525 non-null    float64
 5   gender               525 non-null    object
 6   race                 525 non-null    object
```

```
dtypes: float64(5), object(2)
memory usage: 28.8+ KB
```

In [14]:

```python
# Displayin gthe statistical information of the dataset
data.describe(include='all')
```

Out[14]:

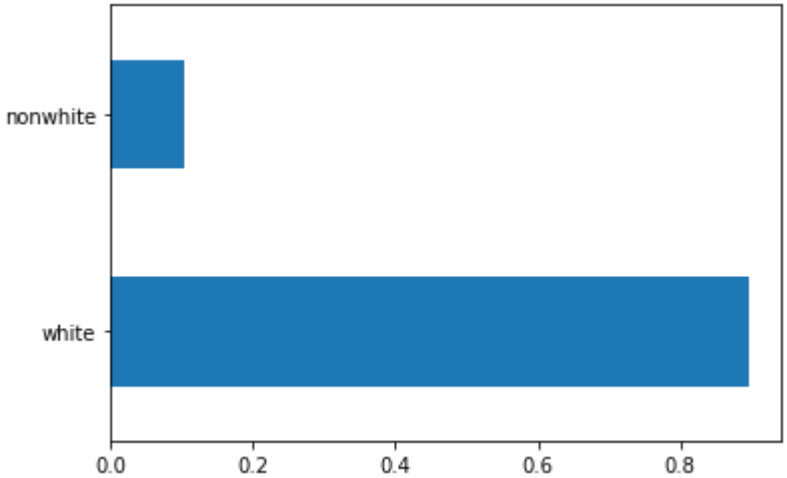
	married	hourly_wage	years_in_education	years_in_employment	num_dependents	gender	race
count	525.000000	525.000000	525.000000	525.000000	525.000000	525	525
unique	NaN	NaN	NaN	NaN	NaN	2	2
top	NaN	NaN	NaN	NaN	NaN	male	white
freq	NaN	NaN	NaN	NaN	NaN	276	471
mean	0.611429	5.917737	12.557471	5.152216	1.044231	NaN	NaN
std	0.487890	3.670713	2.749315	7.215465	1.252465	NaN	NaN
min	0.000000	0.530000	0.000000	0.000000	0.000000	NaN	NaN
25%	0.000000	3.350000	12.000000	0.000000	0.000000	NaN	NaN
50%	1.000000	4.720000	12.000000	2.000000	1.000000	NaN	NaN
75%	1.000000	6.880000	14.000000	7.000000	2.000000	NaN	NaN
max	1.000000	24.980000	18.000000	44.000000	6.000000	NaN	NaN

In [15]:

```python
# counting the number of white and non white race from the data
data.race.value_counts(normalize=True).plot.barh()
```

Out[15]:

```
<AxesSubplot:>
```



The graph above is a frequency distribution graph that shows the distribution of white people and non-white people in the data set. From the graph above, the number of white people in the data set is significantly more than the number of non-white people.

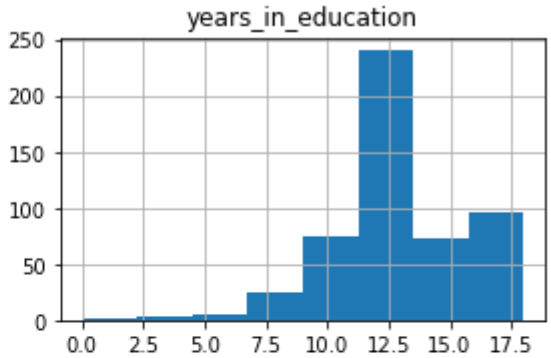
3 Build a graph visualizing the distribution of one or more individual continuous variables of the dataset

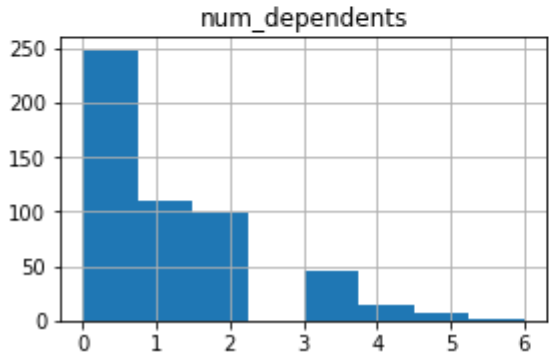
In [16]:

```
# Displaying the histogram for the hourly wage, years in education and number of dependents
data.hist(column=['hourly_wage', 'years_in_education', 'num_dependents'], bins=8, figsize=(10,6))
```

Out[16]:

```
array([[<AxesSubplot:title={'center':'hourly_wage'}>,
        <AxesSubplot:title={'center':'years_in_education'}>],
       [<AxesSubplot:title={'center':'num_dependents'}>, <AxesSubplot:>]],
      dtype=object)
```

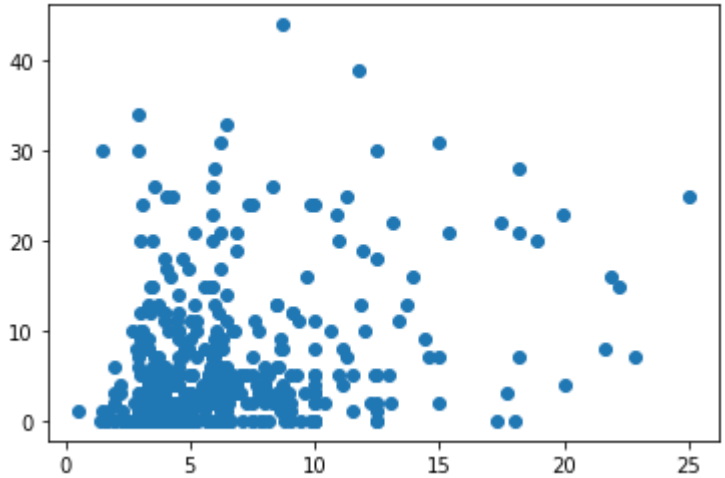




The three graphs above are histograms showing the distributions of the hourly wage, years in education and number of dependents. For the hourly wage distribution, it is seen that a large proportion of people in the data set earn 10 pounds an hour or less. For the years in education distribution, most of the people in the dataset have been in education for 12 years or more. For the number of dependents distribution, most of the demographic, have between 0 and 2 dependents.

In [17]:

```
# Plotting a scatter graph of hourly wage to years in employment
plt.scatter(data.hourly_wage, data.years_in_employment)
plt.show()
```



From the scatter plot above, we see that most of the people earn at most 10 pounds an hour and have been in employment for less than 10 years. The data is aggregated majorly towards the bottom left region of the graph.

4-Building a relationship between a pair of continious variables and determing the correlation between them

In [18]:

```
# Finding the correlation between years in employment and hourly wage
data['years_in_employment'].corr(data['hourly_wage'], method='pearson')
```

Out[18]:

```
0.34097528485222717
```

The correlation between years in employment and hourly wage is just 0.34097528485222717. This implies that both variables are not closely related and little to no information about one can be gotten from the other.

In [19]:

```
# Displaying the count of distribution of white and non-white people
data.race.value_counts()
```

Out[19]:

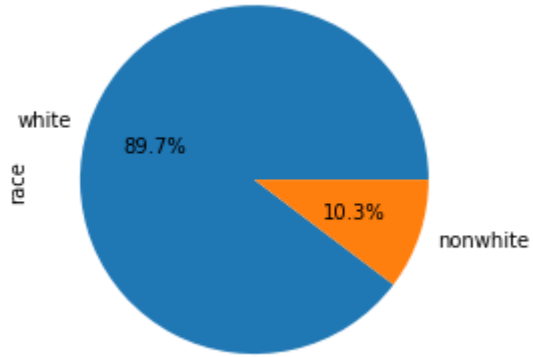
```
white       471
nonwhite     54
Name: race, dtype: int64
```

In [20]:

```
# Plotting a pie chart to show the ratio of white to non-white
```

```
data.race.value_counts().plot.pie(autopct='%1.1f%%')
```

Out[20]: <AxesSubplot:ylabel='race'>



The pie chart above gives a visual representation of what is seen in the count of the racial distribution that the proportion of white people to non-white people is 89.7:10.3.

5 Display unique values of a categorical variable

In [63]:

```
# Displaying the count of distribution of male to female
data.gender.value_counts()
```

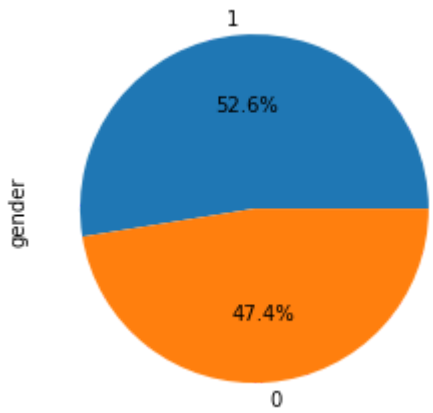
Out[63]:

```
1    276
0    249
Name: gender, dtype: int64
```

In [62]:

```
# Plotting a pie chart to show the ratio of male to female
data.gender.value_counts().plot.pie(autopct='%1.1f%%')
```

Out[62]: <AxesSubplot:ylabel='gender'>



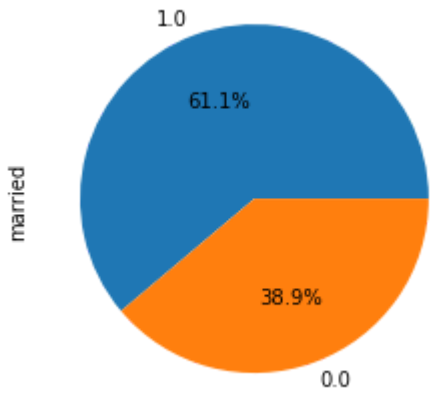
The pie chart shows that the gender distribution of male to female is almost equal as the ratio of male to female is 52.6 to 47.4.

In [23]:
```
# Displaying the count of distribution of married to single
data.married.value_counts()
```

Out[23]:
```
1.0    321
0.0    204
Name: married, dtype: int64
```

In [24]:
```
# Plotting a pie chart to show the ratio of married to single
data.married.value_counts().plot.pie(autopct='%1.1f%%')
```

Out[24]:
```
<AxesSubplot:ylabel='married'>
```



The pie chart above shows that there are more married people than single people in the data set as the ratio of the married to single people is 61.1 to 38.9.

6 Build a contingency table of two potentially related categorical variables. Conduct a statistical test of the independence between them.

In [25]:

```
#  Display a contingency table showing the relationshiop between marital status
# and gender.

contingency_table = pd.crosstab(data['married'], data['gender'])
contingency_table
```

Out[25]:

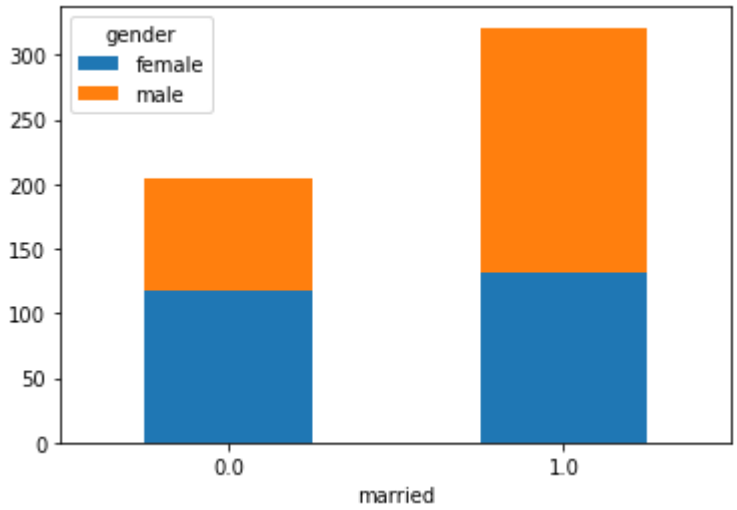
gender	female	male
married		
0.0	118	86
1.0	131	190

In [26]:

```
# Plotting a graph of the contingency table
contingency_table.plot(kind='bar', stacked=True, rot=0)
```

Out[26]:

```
<AxesSubplot:xlabel='married'>
```



In [27]:

```
# Running a chi-squared test on the marital status and gender data
chi2,p_val,dof,expected = stats.chi2_contingency(contingency_table)
print('P value is:', p_val)
```

```
P value is: 0.00019920525782239325
```

After conducting a chi squared test on the marital status and gender, the p-value of 0.001700859727490584 was observed. This is significantly less than the usual significance level of 0.05. Therefore, **we reject the null hypothesis that there is no dependence between the gender and marital status**.

We therefore found evidence that a higher proportion of males are more likely to be married while the females were more likely to be single.

7 Retrieve one or more subset of rows based on two or more criteria and present descriptive statistics on the subset(s).

In [ ]:

```
# Display a subset based on speified criteria.

data1 = data[(data.years_in_employment > 6) & (data.hourly_wage > 6)]
data1
```

There are 68 people in the data set whose experience is above the average years of experience whose hourly wage also exceed the average.

In [29]:

```
# Displaying descriptive statistics on the subset.

data1.describe()
```

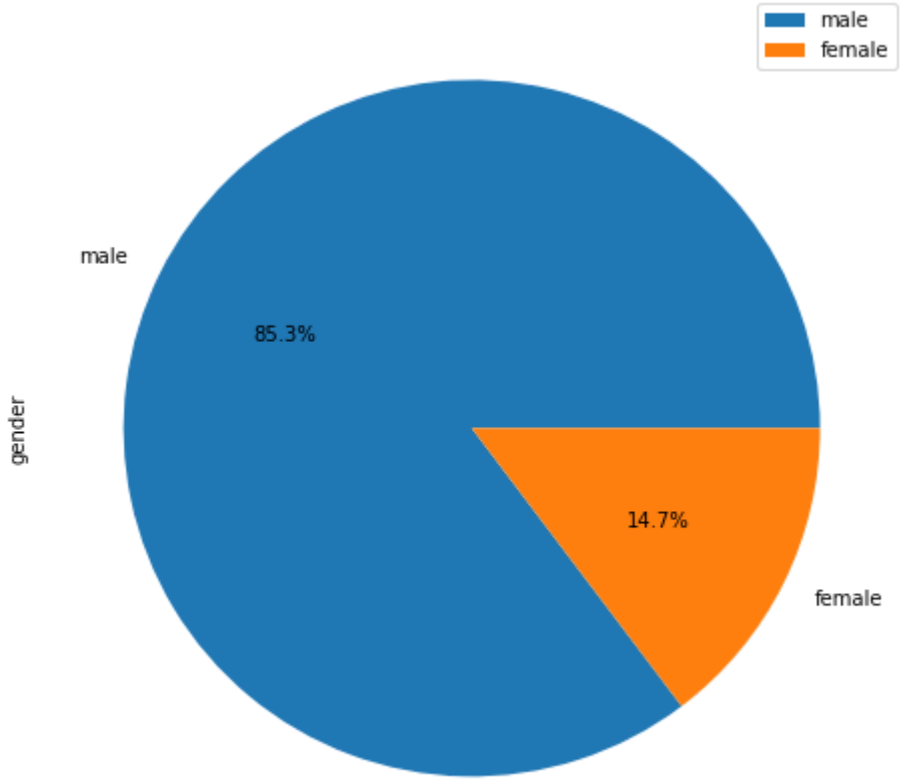
Out[29]:

	married	hourly_wage	years_in_education	years_in_employment	num_dependents
count	68.000000	68.000000	68.000000	68.000000	68.000000
mean	0.808824	11.310882	13.589926	16.308824	1.088886
std	0.396151	4.866664	2.520740	8.454015	1.218420
min	0.000000	6.080000	7.000000	7.000000	0.000000
25%	1.000000	7.465000	12.000000	9.750000	0.000000
50%	1.000000	10.000000	13.000000	13.000000	1.000000
75%	1.000000	13.762500	16.000000	22.000000	2.000000
max	1.000000	24.980000	18.000000	44.000000	4.000000

In [30]:

```
# Plotting a pie chart of the distribution of the genders in the subset
gender_count = data1['gender'].value_counts()
gender_df = pd.DataFrame({'gender': gender_count},
                         index = ['male', 'female'])
gender_df.plot.pie(y='gender', figsize=(8,8), autopct='%1.1f%%')
print(gender_count)
```

```
male      58
female    10
Name: gender, dtype: int64
```



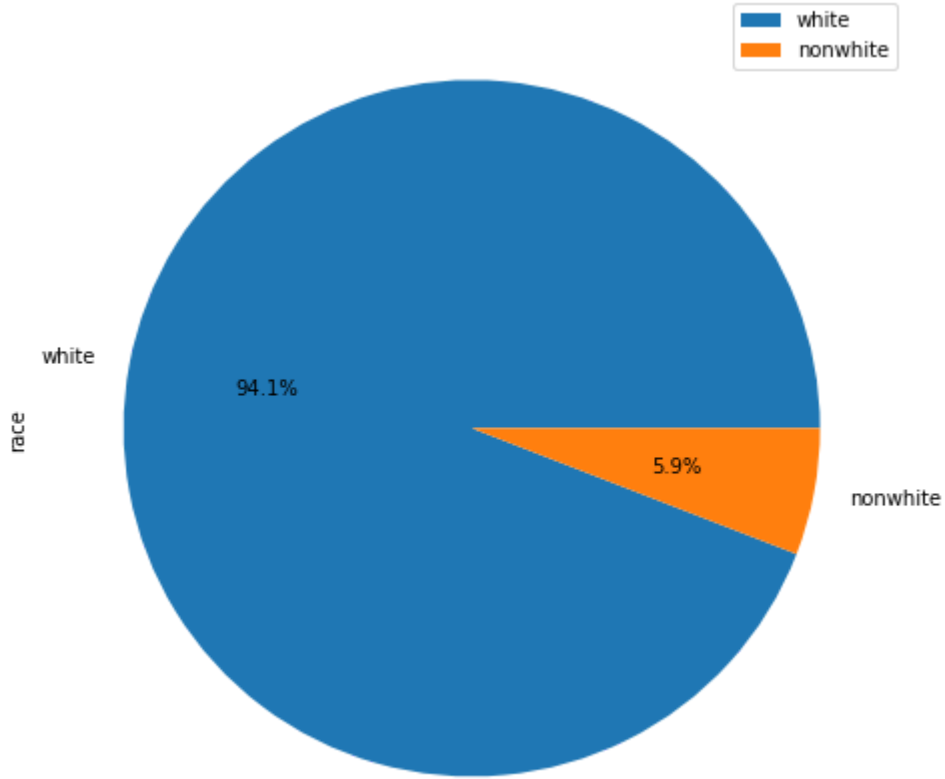
In [31]:

```
# out of 68 people
```

In [32]:

```
# Plotting a pie chart of the distribution of the races in the subset
race_count = data1['race'].value_counts()
race_df = pd.DataFrame({'race': race_count},
                       index = ['white', 'nonwhite'])
race_df.plot.pie(y='race', figsize=(8,8), autopct='%1.1f%%')
print(race_count)
```

```
white       64
nonwhite     4
Name: race, dtype: int64
```



Of the 68 people whose hourly wage rate and years in employment exceeded the average, the gender distribution was 85.3% (or 58 people) being male and 14.7% (or 13 people) being female.

In [34]:

```
# Creating a subset
data2 = data[(data.married==0) & (data.gender=='male')
                 & (data.years_in_education > 13)]
data2.head()
```

Out[34]:

	married	hourly_wage	years_in_education	years_in_employment	num_dependents	gender	race
5	0.0	11.25	18.0	7.0	0.0	male	white
47	0.0	3.65	14.0	2.0	0.0	male	white

	married	hourly_wage	years_in_education	years_in_employment	num_dependents	gender	race
54	0.0	3.25	15.0	0.0	1.0	male	white
58	0.0	4.38	16.0	0.0	0.0	male	white
60	0.0	12.39	14.0	5.0	0.0	male	white

In [35]:

```
# displaying the subset
data2.describe()
```

Out[35]:

	married	hourly_wage	years_in_education	years_in_employment	num_dependents
count	29.0	29.000000	29.000000	29.000000	29.000000
mean	0.0	6.211724	15.793103	2.551724	0.068966
std	0.0	2.974861	1.398451	3.322931	0.257881
min	0.0	2.920000	14.000000	0.000000	0.000000
25%	0.0	4.110000	15.000000	0.000000	0.000000
50%	0.0	5.250000	16.000000	2.000000	0.000000
75%	0.0	7.780000	16.000000	4.000000	0.000000
max	0.0	12.500000	18.000000	13.000000	1.000000

8. Conducting a statistical test of the significance of the difference between the means of the two subsets of the data

In [36]:

```
# Conducts a test to check the significance between the difference of the average wage
# for white and Nonwhite people.
summary, results = rp.ttest(group1= data["hourly_wage"][data['race']=='white'],group1_name ="white",
        group2= data['hourly_wage'][data['race']=='nonwhite'], group2_name ="nonwhite")
print(summary)
```

```
   Variable      N      Mean        SD        SE  95% Conf.  Interval
0     white  471.0  5.948936  3.733307  0.172022   5.610909  6.286962
1  nonwhite   54.0  5.645615  3.087782  0.420194   4.802813  6.488417
2  combined  525.0  5.917737  3.670713  0.160203   5.603018  6.232456
```

In [37]:
```
# printing the results
print(results)
```

```
                 Independent t-test   results
0  Difference (white - nonwhite) =     0.3033
1             Degrees of freedom =   523.0000
2                              t =     0.5748
3          Two side test p value =     0.5657
4         Difference < 0 p value =     0.7172
5         Difference > 0 p value =     0.2828
6                      Cohen's d =     0.0826
7                      Hedge's g =     0.0825
8                  Glass's delta =     0.0812
9                    Pearson's r =     0.0251
```

The p-value of 0.5657 is higher than the confidence interval p-value of 0.05, therefore the null hypothesis is rejected. We can then say that the means of the two subsets of the data are not the same.

9 Create one or more tables that group the data by a certain categorical variable and displays summarized information for each group (e.g. the mean or sum within the group).

In [38]:
```
# Group the data by gender.

gender_group = data.groupby("gender")
gender_group.size()
```

Out[38]:
```
gender
female    249
male      276
dtype: int64
```

In [39]:
```
# Aggregate and summarize info of each variable for the grouped data into a table.

display_grouped = gender_group.agg({"years_in_education":["count","min","max","mean","median","skew"],
                   "years_in_employment":["count","min","max","mean","median","skew"],
                      "hourly_wage":["count","min","max","mean","median","skew"],
                      "num_dependents":["count","min","max","mean","median","skew"]
                     })
display_grouped
```

Out[39]:

	years_in_education	years_in_employment	...	hourly_wage

	count	min	max	mean	median	skew	count	min	max	mean	...	max	mean	median	skew	count	min	max
gender																		
gender	count	min	max	mean	median	skew	count	min	max	mean	...	max	mean	median	skew	count	min	max
female	249	0.0	18.0	12.321285	12.0	-0.884855	249	0.0	34.0	3.725948	...	21.63	4.638419	3.81	2.849898	249	0.0	5.0
male	276	2.0	18.0	12.770552	12.0	-0.574011	276	0.0	44.0	6.438957	...	24.98	7.071904	6.00	1.599322	276	0.0	6.0

2 rows × 24 columns

10. Model Fitting

In [41]:

```
# Encode Categorical Variables as numerical

from sklearn.preprocessing import LabelEncoder
number = LabelEncoder()
data['gender'] = number.fit_transform(data['gender'].astype('object'))
data['race'] = number.fit_transform(data['race'].astype('object'))
data['married'] = number.fit_transform(data['married'].astype('float'))
data.head()
```

Out[41]:

	married	hourly_wage	years_in_education	years_in_employment	num_dependents	gender	race
0	1	3.24	12.0	2.0	3.0	0	1
1	0	3.00	11.0	0.0	2.0	1	1
2	1	6.00	8.0	28.0	0.0	1	1
3	1	5.30	12.0	2.0	1.0	1	1
4	1	8.75	16.0	8.0	0.0	1	1

In [43]:

```
# Fits a model to determine hourly wage using OLS method.

hourly_wage_model = sm.OLS.from_formula(
    "hourly_wage ~ married + years_in_education + years_in_employment + num_dependents + gender + race", data = data).f
hourly_wage_model.summary()
```

Out[43]:

OLS Regression Results

Dep. Variable:	hourly_wage	**R-squared:**	0.350
Model:	OLS	**Adj. R-squared:**	0.342
Method:	Least Squares	**F-statistic:**	46.48
Date:	Fri, 17 Dec 2021	**Prob (F-statistic):**	1.51e-45
Time:	06:25:03	**Log-Likelihood:**	-1314.1
No. Observations:	525	**AIC:**	2642.
Df Residuals:	518	**BIC:**	2672.
Df Model:	6		
Covariance Type:	nonrobust		

	coef	**std err**	**t**	**P>|t|**	**[0.025**	**0.975]**
Intercept	-2.7680	0.760	-3.644	0.000	-4.260	-1.276
married	0.7030	0.283	2.486	0.013	0.148	1.259
years_in_education	0.5214	0.049	10.598	0.000	0.425	0.618
years_in_employment	0.1531	0.019	8.092	0.000	0.116	0.190
num_dependents	0.0957	0.108	0.882	0.378	-0.117	0.309
gender	1.6732	0.268	6.240	0.000	1.146	2.200
race	-0.0665	0.431	-0.154	0.878	-0.914	0.781

Omnibus:	206.380	**Durbin-Watson:**	1.808
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	881.856
Skew:	1.747	**Prob(JB):**	3.22e-192
Kurtosis:	8.302	**Cond. No.**	86.9

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Please discuss more than what you have here, this is just a generic discussion

(1) **Coefficients on the variables**. The estimated coefficients are specified in the second table. Our model is thus described by the line:

$$HourlyWage = -2.6107 + 0.7030 * married + 0.5214 * YearsInEducation + 0.0957 * NumDependennts + 0.1531 * YearsInEmpl$$
.

Considering the signs on the coefficients it is hard to say categorically which of the variables would have the most effect on the hourly wage.

(2) **Significance of the variables**. The p-values on the coefficients indicate that two of the variables which were the Number of dependents and race were insignificant to the model outcome. However, the remaining variables seemed significant with p_values of 0.000 which were less than the significance level of 0.05.

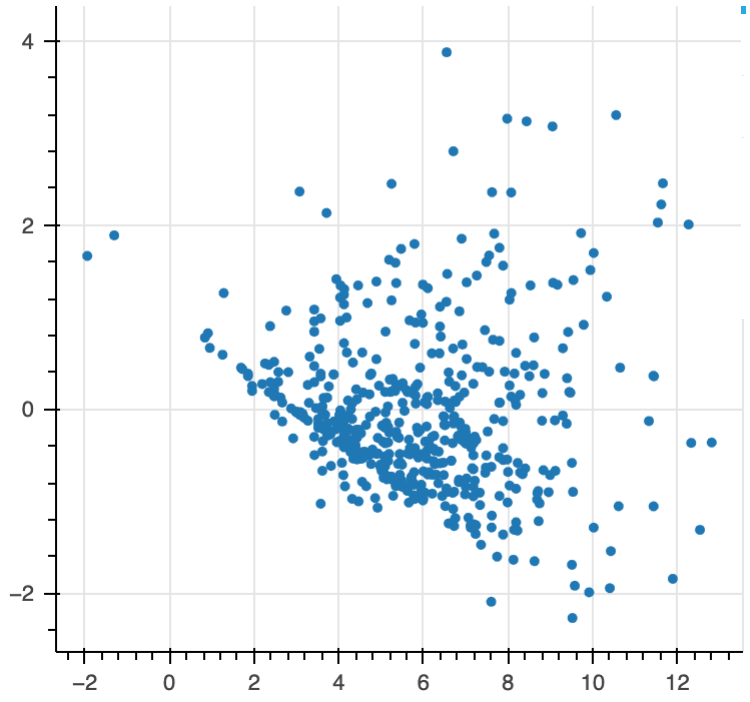
(3) **Quality of the model**. The inital R^2 and the adjusted R^2 values were 0.35 and 0.342 respectively. The two insignificant variables (Number of dependents and race) were removed from the model and the model yielded R^2 and adjusted R^2 values of 0.349 and 0.344 respectively which seemed to further reduce the quality of the model. Hence, we can conclude that there were most likely other factors affecting the hourly wage that our model didn't take into account.

Checking the assumptions of multicollinearity, normality and zero mean of residuals

As with the simple regression model, we can plot the standardized residuals and their histogram to confirm the validity of the assumptions of normality of the distribution of residuals and of the zero mean of residuals are valid with this model.

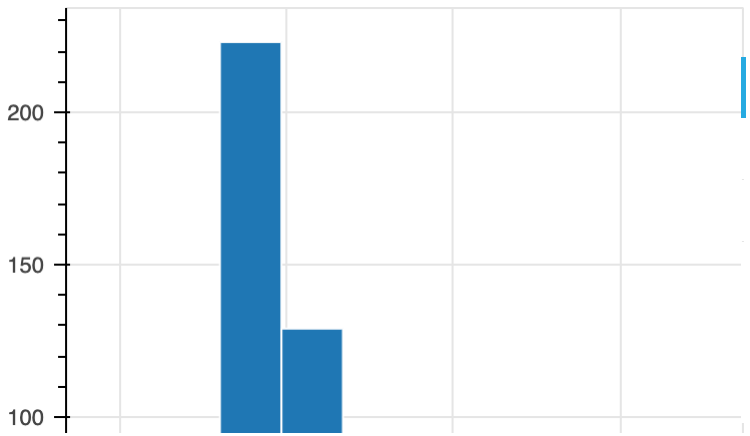
In [44]:

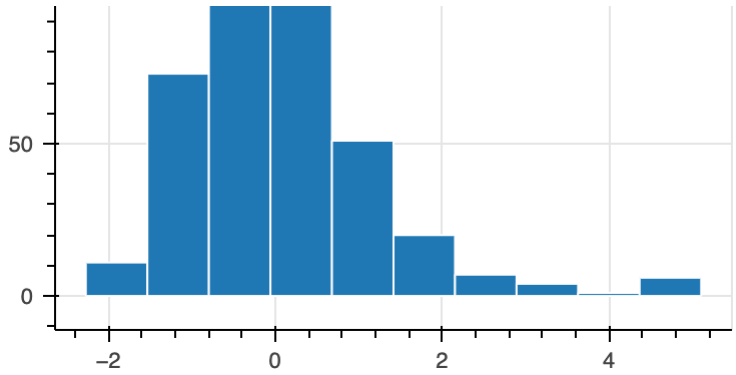
```
fig = figure(height=400, width=400)

# the x axis is the fitted values
# the y axis is the standardized residuals
st_resids = hourly_wage_model.get_influence().resid_studentized_internal
fig.circle(hourly_wage_model.fittedvalues, st_resids)

show(fig)
```



In [45]:

```
# Plot an histogram of 10 bins to show the standardized residual values.

hist, edges = np.histogram(st_resids, bins=10)
res_hist = figure(height=400, width=400)
res_hist.quad(top=hist, bottom=0, left=edges[:-1], right=edges[1:], line_color="white")
show(res_hist)
```





The scatterplot and the histogram suggest the residuals are not equally distributed around 0 and thus not normally distributed. The results of the Jarque-Bera test on the residuals (881.856 - in the third table of the summary) also indicate that the errors are not distributed normally as the value was very high. Also, the p-value equals 3.22e-192 therefore we reject the null hypothesis of normal distribution.

As with the simple linear regression model, we do not test for independence of the errors and for homoskedasticity, as these assumptions are likely to be violated only if the observations are ordered along a temporal dimension.

Thus, the removal of the insignificant variables did not improve the quality of the model, and thus the assumptions of the classical linear regression method did not hold with this particular model.

The method could possibly be improved by addition of other variables/factors we haven't considered due to inavailability in the dataset provided.

In [68]:

```
%%javascript
var nb = IPython.notebook;
var kernel = IPython.notebook.kernel;
var command = "NOTEBOOK_FULL_PATH = '" + nb.notebook_path + "'";
kernel.execute(command);
```

In [69]:

```
import io
from nbformat import read, NO_CONVERT

with io.open(NOTEBOOK_FULL_PATH.split("/")[-1], 'r', encoding='utf-8') as f:
    nb = read(f, NO_CONVERT)
```

```python
word_count = 0
for cell in nb.cells:
    if cell.cell_type == "markdown":
        word_count += len(cell['source'].replace('#', '').lstrip().split(' '))
print(f"Word count: {word_count}")
```

```
Word count: 1080
```

In [ ]: